

não existe qualquer objeto da classe, basta iniciar o nome do dado-membro com o nome da classe e o operador binário de resolução de escopo (::).

- Uma função-membro deve ser declarada como `static` se não acessar dados-membro não `static` ou funções-membro não `static` da classe. Diferentemente das funções-membro não `static`, uma função-membro `static` não possui um ponteiro `this`, pois dados-membro `static` e funções-mem-

bro `static` existem independentemente de quaisquer objetos de uma classe.

### Seção 18.7 Abstração de dados e ocultação de informações

- Tipos de dados abstratos são maneiras de representar noções do mundo real e conceituais em um nível de precisão satisfatório dentro de um sistema de computação.
- Um tipo de dado abstrato captura duas noções: uma representação de dados e as operações que podem ser realizadas nesses dados.

## Terminologia

abstração de dados 560

chamadas de função-membro em cascata 553

composição 543

dequeue (operação de fila) 561

enqueue (operação de fila) 561

estouro aritmético 561

friend, função 549

lista inicializadora de membro 541

objeto host 544

ocultação de informações 560

operações 561

primeiro a entrar, primeiro a sair (FIFO) 561

queue 561

relacionamento *tem-um* 543

representação de dados 561

sintaxe de inicializador de membro 539

`static`, dado-membro 556

`static`, função-membro 557

`this`, ponteiro 551

tipos de dados abstratos (ADT) 561

## Exercícios de autorrevisão

**18.1** Preencha os espaços em cada uma das sentenças:

- \_\_\_\_\_ devem ser usados para inicializar membros constantes de uma classe.
- Uma função não membro precisa ser declarada como um(a) \_\_\_\_\_ de uma classe para ter acesso aos dados-membro `private` dessa classe.
- Um objeto constante precisa ser \_\_\_\_\_; ele não pode ser modificado depois de criado.
- Um dado-membro \_\_\_\_\_ representa informações em escopo de classe.
- As funções-membro não `static` de um objeto têm acesso a um 'ponteiro para si mesmo' para o objeto, chamado ponteiro \_\_\_\_\_.
- A palavra-chave \_\_\_\_\_ especifica que um objeto ou variável não é modificável.
- Se um inicializador de membro não for fornecido a um objeto-membro de uma classe, o \_\_\_\_\_ do objeto é chamado.
- Uma função-membro deve ser `static` se não acessar membros de classe \_\_\_\_\_.
- Objetos-membro são construídos \_\_\_\_\_ do seu objeto de classe delimitador.

**18.2** Encontre os erros nas classes a seguir e explique como corrigi-los:

```
class Example
{
public:
    Example( int y = 10 )
        : data( y )
    {
        // corpo vazio
    } // fim do construtor Example
    int getIncrementedData() const
    {
        return data++;
    } // fim da função getIncrementedData
    static int getCount()
    {
        cout << "Data is " << data << endl;
        return count;
    } // fim da função getCount
private:
    int data;
    static int count;
}; // fim da classe Example
```

## Respostas dos exercícios de autorrevisão

**18.1** a) Inicializadores de membro. b) friend. c) inicializado. d) static. e) this. f) const. g) construtor default. h) não static. i) antes.

**18.2** *Erro:* A definição de classe para `Example` tem dois erros. O primeiro ocorre na função `getIncrementedData`. A função é declarada com `const`, mas modifica o objeto.

*Correção:* Para corrigir o primeiro erro, remova a palavra-chave `const` da definição de `getIncrementedData`.

*Erro:* O segundo erro ocorre na função `getCount`. Essa função é declarada `static`, de modo que não tem permissão para acessar membros não `static` (ou seja, dados) da classe.

*Correção:* Para corrigir o segundo erro, remova a linha de saída da definição de `getCount`.

## Exercícios

**18.3** Explique a noção de ‘amizade’. Explique os aspectos negativos da amizade conforme descritos no texto.

**18.4** Uma definição de classe `Time` correta pode incluir os dois construtores a seguir? Se a resposta for não, justifique.

```
Time( int h = 0, int m = 0, int s = 0 );
Time();
```

**18.5** O que acontece quando um tipo de retorno, até mesmo `void`, é especificado para um construtor ou destrutor?

**18.6** *Modificação da classe Date.* Modifique a classe `Date` na Figura 18.10 para que ela tenha as seguintes capacidades:

a) Saída da data em formatos múltiplos, como

```
DDD AAAA
DD/MM/AA
14 de junho de 1992
```

b) Use construtores sobrecarregados para criar objetos `Date` inicializados com datas nos formatos da parte (a) desse exercício.

c) Crie um construtor de `Date` que leia a data do sistema usando as funções da biblioteca-padrão do cabeçalho `<ctime>` e defina os membros `Date`. (Veja a documentação de referência de seu compilador ou [www.cplusplus.com/ref/ctime/index.html](http://www.cplusplus.com/ref/ctime/index.html) para obter informações sobre as funções no cabeçalho `<ctime>`.)

No Capítulo 19, poderemos criar operadores para testar a igualdade de duas datas e para comparar datas e determinar se uma data vem antes ou depois de outra.

**18.7** *Classe SavingsAccount.* Crie uma classe `ContaPoupanca`. Use um dado-membro `static TaxaJurosAnual` para armazenar a taxa anual de juros para cada um dos correntistas. Cada membro da classe contém um dado-membro `private valorConta`, indicando o valor que o correntista tem atualmente em depósito. Forneça a função-membro `calculaJurosMensais`, que calcula

os juros mensais multiplicando o `balance` por `taxaJurosAnual` dividido por 12; esse juro deverá ser somado a `valorConta`. Forneça uma função-membro `static modificaTaxaJuros` que defina a `static annualInterestRate` como um novo valor. Escreva um programa controlador para testar a classe `ContaPoupanca`. Instancie dois objetos diferentes da classe `ContaPoupanca`, `economia1` e `economia2`, com saldos de R\$ 2.000,00 e R\$ 3.000,00, respectivamente. Defina a `taxaJurosAnual` como 3 por cento. Depois, calcule os juros mensais e imprima os novos saldos de cada um dos correntistas. Depois, defina a `taxaJurosAnual` em 4 por cento, calcule os juros do mês seguinte e imprima os novos saldos de cada um dos correntistas.

**18.8** *Classe InteiroIntervalo.* Crie a classe `InteiroIntervalo` na qual cada objeto pode manter inteiros no intervalo de 0 a 100. Represente o conjunto internamente como um vector de valores `bool`. O elemento `a[i]` é `true` se o inteiro *i* estiver no conjunto. O elemento `a[j]` é `false` se o inteiro *j* não estiver no conjunto. O construtor default inicializa um conjunto para o chamado ‘conjunto vazio’, ou seja, um conjunto no qual todos os elementos contêm `false`.

Forneça funções-membro para as operações de conjunto comuns. Por exemplo, forneça uma função-membro `uniaoConjuntos` que crie um terceiro conjunto que seja a união teórica de conjunto dos dois conjuntos existentes (ou seja, um elemento do resultado é definido como `true` se esse elemento for `true` em um ou ambos os conjuntos existentes, e um elemento do resultado é definido como `false` se esse elemento for `false` em cada um dos conjuntos existentes).

Forneça uma função-membro `intersecaoConjuntos` que crie um terceiro conjunto que seja a intersecção teórica de conjunto dos dois conjuntos existentes (ou seja, um elemento do resultado é definido como `false` se esse ele-

mento for false em um ou ambos os conjuntos existentes, e um elemento do resultado é definido como true se esse elemento for true em cada um dos conjuntos existentes).

Forneça uma função-membro `insereElemento` que coloque um novo inteiro  $k$  em um conjunto definindo `a[k]` como true. Forneça uma função-membro `deletaElemento` que exclua o inteiro  $m$  definindo `a[m]` como false.

Forneça uma função-membro `imprimeConjunto` que imprima um conjunto como uma lista de números separados por espaços. Imprima apenas os elementos que estejam presentes no conjunto (ou seja, sua posição no vetor tem um valor true). Imprima --- para um conjunto vazio.

Forneça uma função-membro `eIguala` que determine se dois conjuntos são iguais.

Forneça um construtor adicional que receba um array de inteiros e o tamanho desse array e use o array para inicializar um objeto definido.

Agora, escreva um programa controlador que teste sua classe `InteiroIntervalo`. Instancie vários objetos `InteiroIntervalo`. Teste se todas as suas funções-membro funcionam corretamente.

**18.9 Modificação da classe Time.** Seria perfeitamente razoável para a classe `Time` das figuras 18.17 e 18.18 representar a hora internamente como o número de segundos desde a meia-noite em vez de os três valores inteiros `hour`, `minute` e `second`. Os clientes devem usar os mesmos métodos `public` e obter os mesmos resultados. Modifique a classe `Time` da Figura 18.17 para implementar a hora como o número de segundos desde a meia-noite, e mostre que não existe mudança visível na funcionalidade para os clientes da classe. [Nota: esse exercício demonstra bem as virtudes da ocultação da implementação.]

**18.10 Embaralhamento e distribuição de cartas.** Crie um programa que embaralhe e distribua cartas. O programa deverá consistir na classe `Card`, na classe `DeckOfCards` e em um programa controlador.

A classe `Card` deverá fornecer:

- a) Dados-membro `face` e `suit` do tipo `int`.
- b) Um construtor que recebe dois `ints` representando o valor e o naipe, e os utilize para inicializar os dados-membro.
- c) Dois arrays `static` de strings representando os valores e os naipes.
- d) Uma função `toString` que retorne a `Card` como uma string na forma '*valor de naipe*'. Você pode usar o operador `+` para concatenar strings.

A classe `DeckOfCards` deverá conter:

- a) Um vector de `Cards` chamado `deck` para armazenar as `Cards`.
- b) Um inteiro `currentCard` representando a próxima carta a ser distribuída.
- c) Um construtor default que inicialize as `Cards` no baralho. O construtor deverá usar a função `vector::push_back` para acrescentar cada `Card` ao final do vector após a `Card` ter sido criada e inicializada. Isso deverá ser feito para cada uma das 52 `Cards` do baralho.
- d) Uma função `shuffle` que embaralhe as `Cards` do baralho. O algoritmo de embaralhamento deverá percorrer o vector de `Cards`. Para cada `Card`, selecione aleatoriamente outra `Card` no baralho e troque as duas `Cards`.
- e) Uma função `dealCard` que retorne o próximo objeto `Card` do baralho.
- f) Uma função `moreCards` que retorne um valor `bool` indicando se existem mais `Cards` a serem distribuídas.

O programa controlador deverá criar um objeto `DeckOfCards`, embaralhar as 52 cartas e depois distribuí-las.

**18.11 Embaralhamento e distribuição de cartas.** Modifique o programa que você desenvolveu no Exercício 18.10 para que ele trate de uma mão de pôquer de cinco cartas. Depois, escreva funções para realizar cada uma das tarefas a seguir:

- a) Determinar se a mão contém um par.
- b) Determinar se a mão contém dois pares.
- c) Determinar se a mão contém três cartas iguais (por exemplo, três valetes).
- d) Determinar se a mão contém quatro cartas iguais (por exemplo, quatro ases).
- e) Determinar se a mão contém um *flush* (ou seja, todas as cinco cartas do mesmo naipe).
- f) Determinar se a mão contém um *straight* (ou seja, cinco cartas de valores consecutivos).

### Projetos de embaralhamento e distribuição de cartas

**18.12 Embaralhamento e distribuição de cartas.** Use as funções do Exercício 18.11 para escrever um programa que lide com duas mãos de pôquer com cinco cartas, avalie cada uma e determine qual é a melhor.

**18.13 Embaralhamento e distribuição de cartas.** Modifique o programa que você desenvolveu no Exercício 18.12 de modo que possa simular o distribuidor das cartas. A mão de cinco cartas do distribuidor é distribuída 'virada para baixo', para que o jogador não possa vê-la. O programa deverá, então, avaliar a mão do distribuidor e, com base na qualidade da mão, retirar uma, duas ou três outras cartas para substituir

o número correspondente de cartas desnecessárias na mão original. Então, o programa deverá reavaliar a mão do distribuidor.

**18.14 Embaralhamento e distribuição de cartas.** Modifique o programa que você desenvolveu no Exercício 18.13 para que ele trate da mão do carteador, mas de maneira que o jogador possa decidir quais cartas de sua mão deverão ser substituídas. O programa deverá, então, avaliar as

duas mãos e determinar quem ganhará. Agora, use esse novo programa para jogar 20 partidas contra o computador. Quem ganha mais partidas, você ou o computador? Peça a um de seus amigos que jogue 20 partidas contra o computador. Quem ganha mais partidas? Com base nos resultados dessas partidas, faça as modificações apropriadas para refinar o programa de pôquer. Jogue mais 20 partidas. Seu programa modificado funciona melhor?

## Fazendo a diferença

**18.15 Projeto de controle de tráfego aéreo.** Todos os dias, de acordo com a National Air Traffic Controllers Association (<[www.natca.org/mediacenter/bythenumbers.msp](http://www.natca.org/mediacenter/bythenumbers.msp)>), mais de 87 mil voos decolam nos Estados Unidos, incluindo voos comerciais, transporte de carga, e outros, e a tendência a longo prazo é que a atividade de tráfego aéreo cresça juntamente com a população. Com o crescimento do tráfego aéreo, aumentam também os desafios dos controladores que monitoram os voos e oferecem instruções aos pilotos para garantir a segurança no céu.

Nesse exercício, você criará uma classe `Voo` que poderá ser usada em um simulador de controle de tráfego aéreo simples. A função `main` da aplicação atuará como controle de tráfego aéreo. Visite sites como

<[www.howstuffworks.com/air-traffic-control.htm](http://www.howstuffworks.com/air-traffic-control.htm)>

para descobrir como funciona o sistema de controle de tráfego aéreo. Depois, identifique alguns dos principais atributos de um `Voo` em um sistema de controle de tráfego aéreo. Pense nas diferentes circunstâncias em que um avião poderia voar desde o momento em que está estacionado no portão de um aeroporto até o momento em que chega a seu destino — estacionado, taxiando, esperando para decolar, decolando, subindo etc. Use uma enumeração `StatusVoo` para representar esses estados. Os atributos poderiam incluir a marca e o modelo do avião, a velocidade do ar, altitude, direção, transportadora, horário da decolagem, horário aproximado do pouso, origem e destino. A origem e o destino devem ser especificados usando códigos de aeroporto de três letras, como BOS para Boston e LAX para Los Angeles (esses códigos estão disponíveis em <[world-airport-codes.com](http://world-airport-codes.com)>). Forneça funções `set` e `get` para manipular esses e quaisquer outros atributos que você identifique. Em seguida, identifique os comportamentos da classe e implemente-os como funções da classe. Inclua comportamentos como `mudaAltitude`, `reduzVelocidade` e `iniciaPouso`. O construtor `Voo` deverá inicializar os atributos de um `Voo`. Você também deverá fornecer uma função `toString` que retorne uma representação em `string` do

status atual de um `Voo` (por exemplo, estacionado no portão, taxiando, decolando, mudando de altitude). Essa `string` deverá incluir todos os valores de variável de instância do objeto.

Quando a aplicação for executada, `main` mostrará a mensagem “SimulacaoControleTrafegoAereo”, e depois criará e interagirá com três objetos `Voo` que representem aviões que estão atualmente voando ou preparando-se para voar. Para simplificar, a confirmação do `Voo` de cada ação será uma mensagem exibida na tela quando a função apropriada for chamada ao objeto. Por exemplo, se você chamar a função `mudaAltitude` de um `voo`, o método deverá:

- a) Mostrar uma mensagem que contenha o nome da companhia aérea, o número do voo, “mudaAltitude”, a altitude atual e a nova altitude.
- b) Mudar o estado do dado-membro `status` para `MUDA_ALTITUDE`.
- c) Mudar o valor do dado-membro `novaAltitude`.

Em `main`, crie e inicialize três objetos `Voo` que estejam em diferentes estados — por exemplo, um poderia estar no portão, um poderia estar se preparando para decolar e um poderia estar se preparando para aterrissar. A função `main` deverá enviar mensagens para (invocar funções em) os objetos `Voo`. À medida que um objeto `Voo` recebe cada mensagem, ele deve mostrar uma mensagem de confirmação da função sendo chamada — como “[Nome da companhia] [Número do voo] mudando a altitude de 20.000 para 25.000 pés”. A função também deverá atualizar a informação de circunstância apropriada no objeto `Voo`. Por exemplo, se o Controle de Tráfego Aéreo enviar uma mensagem como “[Companhia] [número do voo] descendo para 12.000 pés”, o programa deverá executar uma chamada de função como `voo1.mudaAltitude(12000)`, que mostraria uma mensagem de confirmação e definiria o dado-membro `novaAltitude` como 12000. [Nota: suponha que o dado-membro `altitudeAtual` de `Voo` seja definido automaticamente pelo altímetro do avião.]